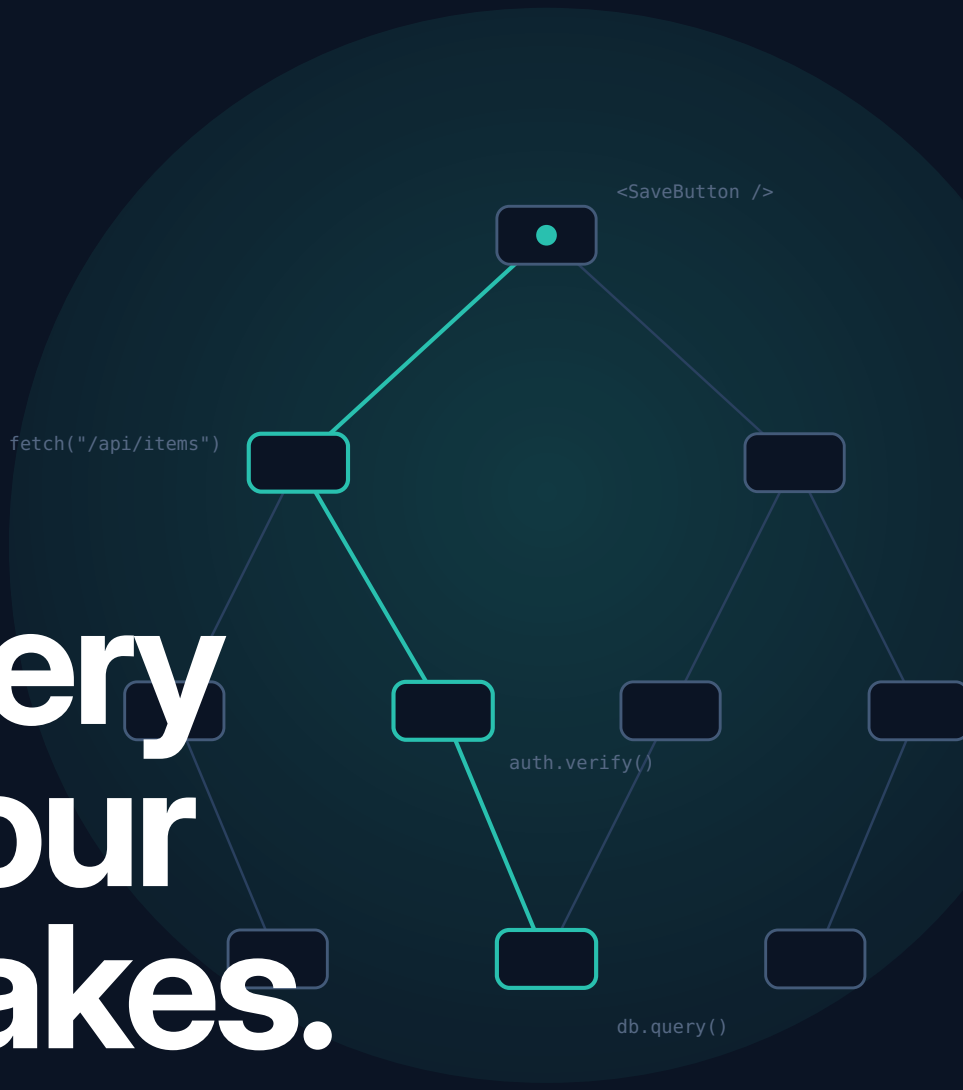
LANTERN · WORKING TITLE



See every path your code takes.

Product specification for a full-stack code-mapping, explanation, and deployment-readiness platform built for AI-assisted developers.

DOCUMENT	DATE	OWNER	STATUS
Product Specification Version 0.2 · Draft	October 30, 2026	Little Henderson	For review · Not for distribution

CONTENTS

What this document covers

01	Executive summary	03
02	Problem, audience, and positioning	05
03	Product principles	07
04	Feature overview	08
05	System architecture and the two access modes	10
06	Analysis engine	13
07	The Three-Lens explanation system	17
08	Model choice: the multi-LLM layer	20
09	Security audit	22
10	Deployment readiness and systems design	25
11	The Evidence Report	27
12	The PWA client	28
13	Technology stack, data model, and API	30
14	Trust and security of Lantern itself	32
15	Non-functional requirements	33
16	Roadmap	34
17	Risks, open questions, and success metrics	36

Document control

Working title	Lantern. A placeholder; trademark and domain checks are pending.
Scope	Product, architecture, and phased delivery for v1. Pricing and go-to-market are sketched, not decided.
Conventions	"MUST", "SHOULD", and "MAY" mark requirement strength. Performance numbers are targets to validate in Phase 0, not measured results.
Changes in v0.2	Lantern is now full stack: TypeScript/JavaScript joins Python in Phase 1, ahead of Rust and C++, and a bridge links each frontend API call to the backend route that answers it (Section 6.4). Scanners, security suggestions, roadmap, and risks updated; Phase 0 results added (16.1).
Third-party tools	Named tools reflect the landscape as of October 2026. Licenses and capabilities change, so each choice gets re-checked before build.

01 · EXECUTIVE SUMMARY

Executive summary

Lantern maps every call path in a codebase, explains each piece in three registers, audits it with real security tooling, and grades how ready it is to ship. The output is evidence a builder can hand to a skeptical engineer.

A new class of builders ships software with AI assistance. They move fast, but they often can't explain what their code does, and they don't know what they don't know about security or deployment. Professional engineers notice, and the label "AI slop" sticks even when the work is sound.

Lantern closes that gap in four moves:

1. **Map.** It parses Python and TypeScript/JavaScript repositories (then Rust and C++) into a single call graph, joins the frontend to the backend wherever one calls the other over HTTP, and renders it as an interactive flowchart. A path starts at a real entry point, such as a page, a button's click handler, an API route, `main()`, or a CLI command, and can run unbroken from the button to the database.
2. **Explain.** Every function and module gets three explanations: a technical one, a plain-English one with analogies, and a mapped one where each everyday phrase is pinned to the exact technical term it stands for.
3. **Audit.** Established static-analysis and dependency scanners find the security problems. The call graph tells you which findings are actually reachable. An LLM of the user's choosing explains each finding and drafts a fix, and a rescan verifies the fix.
4. **Prove.** A Deployment Readiness Score and a shareable Evidence Report tie each claim to a specific commit and to the tool that produced it.

TWO WAYS IN

Cloud mode

Connect GitHub. A sandboxed worker clones the repo, analyzes it, and destroys the copy. Zero setup, works from any device.

TWO WAYS IN

Local agent mode

Run one binary on your own machine. Source never leaves it; only results are uploaded, with optional redaction. Model calls can stay local too.

THE CORE DESIGN BET

Deterministic tools find facts; language models explain them. The call graph, the security findings, and the readiness checks come from parsers and scanners whose output is repeatable. Models never invent structure or vulnerabilities. They translate verified facts into language people can learn from. That separation is what makes the output credible to engineers.

v1 scope at a glance

AREA	IN V1	LATER
Languages	Python and TypeScript/JavaScript, linked full stack (Phase 1); Rust (Phase 2); C++ (Phase 3)	Go, Java, Kotlin, Swift
Hosted backends	Calls to the app's own API routes, in any supported language	Supabase, Firebase, and similar client SDKs modeled as database edges, with their access rules checked
Access	GitHub cloud mode; local agent on macOS, Linux, Windows	GitLab, Bitbucket, self-hosted cloud workers
Models	Bring-your-own-key for major providers, plus local models	Hosted credits, fine-tuned explainer model
Client	Installable PWA with offline report viewing	IDE extensions, native wrappers
Collaboration	Single user, shareable read-only reports	Teams, comments, review workflows

02 · PROBLEM AND AUDIENCE

Problem, audience, and positioning

AI tools let people build software faster than they can understand it. Lantern sells understanding, and the proof of it.

2.1 The gap

An AI-assisted builder can produce a working app in a weekend. What they usually can't produce is an answer to the questions a reviewer asks: *What calls this? What happens if this input is malicious? Where are the secrets? How does this run in production? What breaks under load?* The criticism they get is often fair on the specifics, but it lands as dismissal of the whole person. Nothing on the market today turns "I built this" into "I understand this, and here's the evidence."

Most of these apps are full stack: a React or Next.js frontend, usually in TypeScript, talking to an API or a hosted database. Many of the holes engineers point to live exactly where the two halves meet: server secrets shipped in the browser bundle, API routes the frontend calls without any login check, database access rules switched off, and CORS left wide open. A reviewer's real question is usually *"what happens when someone clicks this button?"* Answering it means following one path across both halves, so Lantern treats the whole stack as one program.

2.2 Who it's for

PRIMARY

The AI-assisted builder

Ships real projects with AI coding tools. Self-taught or career-switching. Wants to learn what their code does and stop getting dismissed. Uses a phone and laptop interchangeably.

Job to be done: "Help me understand my code well enough to defend it, and fix what's indefensible."

SECONDARY

The reviewing engineer

Receives an Evidence Report from a builder, a contractor, or a junior teammate. Has five minutes. Trusts scanners more than claims.

Job to be done: "Show me, quickly and verifiably, whether this is safe to merge or deploy."

SECONDARY

The learner and educator

Students, bootcamps, and self-learners who use the Three-Lens view to climb from plain English to real terminology on codebases they care about.

FUTURE

The small team lead

Onboards new hires onto an unfamiliar codebase and wants a living map plus a readiness gate in CI.

2.3 Landscape

CATEGORY	WHAT IT DOES WELL	WHAT'S MISSING FOR THIS AUDIENCE
Call-graph tools	Accurate structure (code2flow, pyan, IDE call hierarchies)	No explanation, no security, no shareable output; developer-only UX
AI repo explainers	Fluent summaries of a whole repository	Structure is inferred by the model, so it can be wrong; no plain-to-technical mapping
Security scanners	Trusted findings	Output assumes security expertise; little guidance on reachability or fixes for beginners
AI code review bots	PR-level feedback	Diff-focused, not whole-system understanding; no learning path

POSITIONING STATEMENT

For AI-assisted builders who get dismissed by engineers, Lantern is the code-understanding platform that turns a repository into a verified map, plain-English explanations, and a security and readiness report. Unlike AI explainers, every structural and security claim comes from deterministic tools and is traceable to a commit.

03 · PRINCIPLES

Product principles

These principles settle design arguments. When two good ideas conflict, the higher one wins.

PRINCIPLE	WHAT IT MEANS IN PRACTICE
1 · Facts from tools, words from models	Call edges, findings, and readiness checks MUST come from deterministic analysis. Models MUST only explain, triage, and propose fixes, and their output is always labeled as model-generated.
2 · The user decides where code goes	Cloud mode and local mode are equal first-class paths. Every screen that sends data anywhere says exactly what is sent and to whom.
3 · Show confidence, never fake certainty	Every edge and finding carries a confidence level. Dynamic languages produce uncertain graphs; Lantern says so instead of hiding it.
4 · Teach, don't just tell	Plain language is a ramp toward real terminology, never a replacement for it. The mapped lens exists so learners graduate.
5 · Evidence must be verifiable	Reports are pinned to a commit hash, list the tools and versions used, and state what they do not prove.
6 · Practice what we preach	Lantern handles source code and API keys. It MUST pass its own audit and publish its security posture.
7 · Mobile is a real surface	Every core task, short of initial local-agent setup, works on a phone.

> **TONE GUARDRAIL**
>
> Lantern is on the builder's side but never flatters. A low readiness score is shown plainly, with the path to improve it. The report's credibility with engineers depends on it being honest with builders.

04 · FEATURES

Feature overview

Six capabilities, each built on the one before it.

F1

Code Atlas

An interactive map of every function, method, and module, and who calls whom. Two views: a zoomable network overview, and a Flow view that lays out one entry point's call path as a top-down flowchart.

F2

Three-Lens explanations

Technical, Everyday, and Mapped explanations for every node, toggled instantly. The Mapped lens pairs each plain phrase with its technical twin, one-to-one.

F3

Security audit

Real scanners, reachability analysis over the call graph, model-written explanations in all three lenses, suggested fixes as diffs, and verification by rescan. Also proactive suggestions for security features the app lacks.

F4

Deployment readiness

A 0–100 score across ten categories, plus a systems-design advisor that recommends a deployment shape and generates starter files such as a Dockerfile, a CI workflow, and `.env.example`.

F5

Evidence Report

A shareable, commit-pinned report with a public link and PDF export, built for an engineer with five minutes.

F6

Model choice

Bring your own API key for any major provider, or run local models. Pick models per task, preview costs before a run, and compare two models' explanations side by side.

4.1 Core user journey

1. **Connect.** The user signs in on the PWA and chooses cloud mode (authorize the GitHub App on selected repos) or local mode (install the agent and pair it by scanning a QR code or entering a one-time code).
2. **Configure.** They choose models for each task and see an estimated token cost for a full run. They choose a privacy level (Section 5.3).
3. **Analyze.** Structural analysis and scanning run first and finish quickly. Explanations stream in afterward, entry points first. A push notification fires when the run completes.
4. **Explore.** They open the Atlas at an entry point, follow the path, and toggle lenses on any node.

5. **Fix.** They work down the findings and readiness checklist, apply suggested patches (cloud mode opens a pull request; local mode writes a patch file or applies it on confirmation), then re-run.
6. **Prove.** They publish an Evidence Report and share the link.

05 · ARCHITECTURE

System architecture and the two access modes

One analysis engine, packaged as a single binary, runs in two places. That guarantees identical results whether the user picks cloud or local.

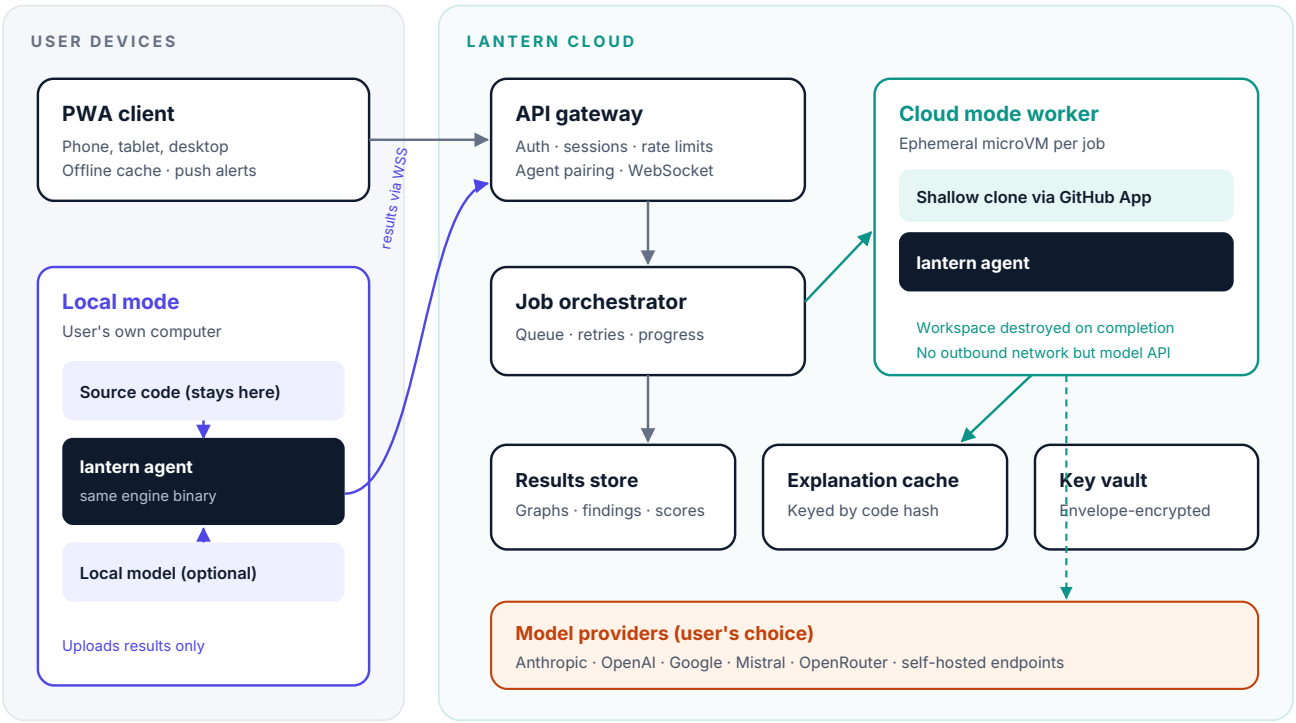


Figure 1. System overview. The same `lantern agent` binary performs analysis in both modes. In cloud mode it runs inside a disposable sandbox; in local mode it runs on the user's machine and sends only results.

5.1 Cloud mode

Best for zero setup and for users who work mostly from a phone or tablet.

- **Access.** A GitHub App with the narrowest permissions: read-only repository contents and metadata on repositories the user explicitly selects. Pull-request write permission is requested separately, only when the user first asks Lantern to open a fix PR.
- **Isolation.** Each job runs in a fresh microVM (Firecracker or gVisor class) with no persistent disk, CPU and memory caps, a wall-clock limit, and egress restricted to the configured model endpoints.
- **Untrusted code.** The worker never runs the user's build scripts or tests by default. This matters most for C++ (CMake configure steps execute code), Rust (build scripts and procedural macros execute code), and JavaScript (`npm install` runs package lifecycle scripts, so the worker never installs dependencies; it reads lockfiles instead). Section 6.3 covers the accuracy trade-off and the opt-in "sandboxed build" setting.

- **Retention.** The clone is destroyed when the job ends. Only derived results persist: the graph, findings, scores, and explanations. Source snippets inside results are governed by the privacy level.

5.2 Local agent mode

Best for private or proprietary code and for users who want model calls to stay on their own hardware.

- **Distribution.** A single signed binary for macOS (Apple silicon and Intel), Linux (x86-64 and ARM64), and Windows. Install via Homebrew, a shell script with a published checksum, winget, or `cargo install`.
- **Pairing.** `lantern login` shows a one-time code and QR code. The user approves it in the PWA, and the agent receives a device-scoped, revocable token.
- **Operation.** `lantern analyze` runs once. `lantern watch` re-analyzes on file changes. The agent can also sit idle and accept "analyze now" requests triggered from the PWA, so a user on their phone can kick off a run on their home machine.
- **Model calls.** Made directly from the agent using keys stored in the operating system's keychain, or sent to a local model server. In this mode Lantern's servers never see API keys or source.
- **Tooling.** The agent uses language servers and scanners already installed on the machine when present, and offers to fetch pinned versions when they're missing.

5.3 What leaves the machine: privacy levels

Both modes honor one setting that controls how much source-derived content is stored in Lantern's cloud. The PWA shows the active level on every analysis.

LEVEL	STORED IN LANTERN CLOUD	TRADE-OFF
Full	Graph, findings, explanations, and short source snippets (the function body shown beside its explanation)	Best experience; code viewable from any device
Structure only	Graph, symbol names, file paths, findings, explanations. No source text.	Code panel only works when the agent is online
Redacted	As above, with names and paths replaced by stable aliases. The alias map stays on the user's machine.	Shared reports show aliases unless the user publishes the map
Offline	Nothing. The agent serves the PWA over the local network or localhost.	No remote access or sharing

> **BE PRECISE WITH USERS**
> Even in local mode, if the user chooses a cloud model provider, source code is sent to that provider to generate explanations. The UI MUST say this in plain words when a cloud model is selected, and MUST offer local models as the alternative. "Your code never leaves your computer" is only true with a local model.

5.4 Mode comparison

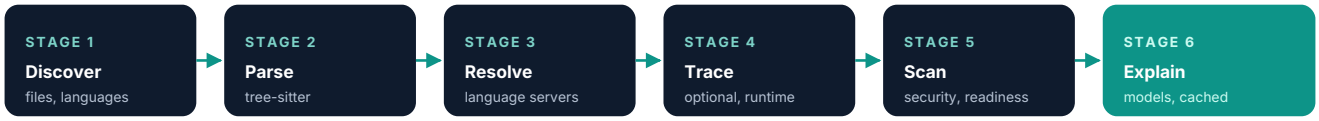
CONSIDERATION	CLOUD MODE	LOCAL AGENT MODE
Setup	Two taps: sign in, select repos	Install binary, pair once
Where source lives	Briefly in a disposable sandbox	Never leaves the machine (except to a chosen cloud model)
Analysis accuracy	Good; build-dependent analysis limited by default	Best; can use the user's real build environment
Runtime tracing	Opt-in, test suite only, in sandbox	Full: run the app or tests and record real calls
Private or non-GitHub repos	GitHub only in v1	Any folder, any host
API key custody	Encrypted in Lantern's vault	OS keychain; never sent to Lantern
Applying fixes	Opens a pull request	Writes a patch, or applies on confirmation

06 · ANALYSIS ENGINE

Analysis engine

The engine turns a repository into a Unified Code Graph: one language-neutral model of every symbol and every call, with an honest confidence level on each edge.

6.1 Pipeline



Stages 1–5 are deterministic and run in seconds to minutes. Stage 6 is the only stage that calls a model, and it streams results as they arrive.

Figure 2. Analysis pipeline. Each stage writes to the Unified Code Graph; later stages enrich it rather than replace it.

6.2 Unified Code Graph

All languages are normalized into one graph schema, so the PWA, the scanners' reachability step, and the explainer never need language-specific logic.

NODE KIND	EXAMPLES
Module	Python module, TS/JS file, Rust crate or module, C++ translation unit
Type	Class, struct, trait, enum, interface
Callable	Function, method, closure, lambda, arrow function, React component
EntryPoint	`main`, page, route handler, CLI command, test
External	Third-party library call, system call, network or database boundary

EDGE KIND	MEANING
calls	A invokes B
calls (http)	Frontend code calls a backend route over the network (Section 6.4)
may_call	A invokes one of a set (dynamic dispatch), or A hands B over to run later (callback, event handler)
contains	Module or type owns a member
implements	Type fulfills a trait or base class
reads / writes	Touches a file, environment variable, socket, or database

Edge confidence

Every edge carries one of four confidence levels, displayed in the Atlas as line style so users can see at a glance what is proven and what is inferred.

LEVEL	LINE STYLE	SOURCE
Observed	Solid, bold	Recorded during a real run (Stage 4)
Resolved	Solid	A language server or compiler-grade resolver confirmed the target
Inferred	Dashed	Name and type heuristics; plausible but unconfirmed
Possible	Dotted	One of several candidate targets of dynamic dispatch

6.3 Language support

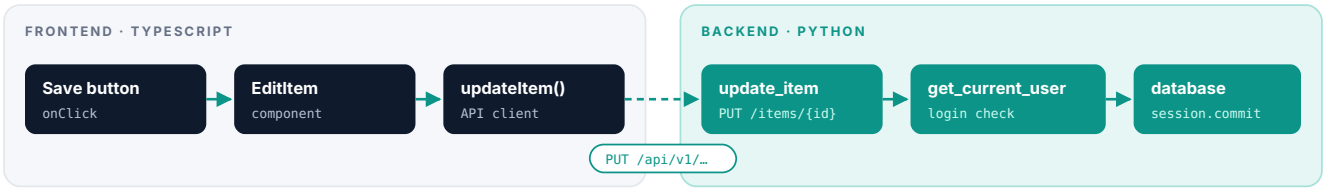
LANGUAGE	PARSE AND RESOLVE	HARD CASES	RUNTIME TRACING
Python	tree-sitter for syntax; Pyright (or an equivalent type-aware analyzer) for resolution. Import graph built from the project's real layout and virtual environment when available.	Monkey-patching, `getattr`, decorators that wrap callables, framework magic (Django signals, FastAPI dependency injection). Framework adapters recognize common patterns.	PEP 669 `sys.monitoring` on Python 3.12+, with a `sys.setprofile` fallback for older versions. Low overhead, records caller→callee pairs only.
TypeScript / JavaScript	tree-sitter (TS, TSX, JS grammars) with an import resolver that follows re-exports, tsconfig `paths`, and bundler aliases; the TypeScript compiler API or tsserver for type-aware resolution in Phase 1.	Hooks and closures, objects passed around as props, callbacks and event handlers (shown as *possible* because they run later), dynamic imports, framework conventions (file-based routing, server actions). Framework adapters cover React, Next.js, React Router and TanStack Router, Express, Hono, and Fastify.	Opt-in V8 coverage or a Node `--require` hook for server code, and a browser-side recorder driven by Playwright for frontend flows; local mode only in v1.
Rust	tree-sitter plus rust-analyzer for name and trait resolution.	Trait objects (`dyn Trait`) become *may_call* sets. Macros are expanded where rust-analyzer can. In cloud mode build scripts and procedural macros are disabled by default because they execute arbitrary code, which lowers accuracy on macro-heavy crates.	Opt-in instrumentation through the `tracing` ecosystem or a sampling profiler; local mode only in v1.
C++	tree-sitter for a fast first pass; clangd driven by `compile_commands.json` for accurate resolution. The agent can generate the compile database from CMake, Meson, or Bear.	Templates, macros, virtual calls, function pointers, and build-flag-dependent code. Without a compile database, accuracy drops sharply; the UI says so prominently.	Opt-in compiler instrumentation (`-finstrument-functions`) or uprobes-based tracers on Linux; local mode only in v1.

WHY THE ORDER IS PYTHON → TYPESCRIPT/JAVASCRIPT → RUST → C++

Python has the simplest toolchain and a large share of AI-built backends. TypeScript/JavaScript comes right after because most apps this audience ships are full stack, and a Python-only map leaves the frontend, often most of the code, invisible: on the FastAPI full-stack template, 10,000 of its 13,000 lines are TypeScript. Rust has excellent tooling and moderate difficulty. C++ is the hardest to analyze correctly and the riskiest to analyze safely in the cloud, so it ships last, with lessons learned from the rest.

6.4 Full-stack tracing: the frontend-to-backend bridge

The signature output of a full-stack Lantern is one unbroken path through the whole app. Each language is parsed into the same graph, and then a bridge stage joins them: every HTTP call found in frontend code is matched to the backend route that answers it.



The dashed hop is the bridge: a network call matched to its route by method and path. Every other hop is an ordinary call edge.

Figure 2b. One path from a button to the database, measured on the open-source FastAPI full-stack template in Phase 0.

What the bridge reads

- **Frontend calls:** `fetch`, axios-style clients (`api.get("/items")`), and generated API clients that carry a `url` field. Template strings become path parameters, so `` `/items/${id}` `` matches `/items/{id}`.
- **Backend routes:** every route entry point in the graph, from any language: FastAPI, Flask, and Django in Python; Next.js route handlers, Express, Hono, and Fastify in JavaScript.
- **Third-party APIs:** calls that match no route stay on the map as External nodes with their host, so the user sees where data leaves the app.

Match confidence

MATCH	CONFIDENCE	RULE
Exact	Resolved	Same method and the same path segments, with parameters lining up
Shared prefix	Inferred	The route's path is the tail of the call's path, because the backend mounts its routers under a prefix it reads at runtime (for example `/api/v1` from a setting). Literal segments outrank parameters, so `/users/me` beats `/users/{id}`.
Tie	Possible	Two or more routes match equally well. Lantern links each one as possible and never picks silently.

> **WHY THIS MATTERS FOR SECURITY**
> Once the frontend and backend share one graph, reachability (Section 9.2) runs across the network boundary. A finding deep in the backend can be traced to the exact page and button that reach it, and a route the frontend calls with no login check on its path stands out on the map.

6.5 Entry-point detection and scale

Large repositories produce graphs too dense to read. Lantern never opens on the whole graph. It detects entry points and starts there:

- **Detected automatically:** `if __name__ == "__main__"`, `fn main`, `int main`, console-script and binary targets in `pyproject.toml`, `package.json`, `Cargo.toml`, and CMake; route decorators for Flask, FastAPI, Django, Axum, and Actix; Next.js pages and route handlers, React Router and TanStack Router file routes, and Express-style `app.get("/path", handler)` servers; app starts such as `createRoot` and `.listen()`; CLI frameworks such as Click, Typer, argparse, and clap; and test functions.
- **Pages and buttons:** frontend entry points are pages, and event handlers wired in JSX (`onClick`, `onSubmit`) appear as the first hops beneath them, so a user can start from "what happens when someone clicks Save?"
- **Collapsing:** modules collapse into single nodes until expanded. Calls into the standard library and third-party packages collapse into an "External" boundary node by default.
- **Incremental runs:** each callable is fingerprinted by a hash of its normalized syntax tree. Re-runs only re-resolve changed files and only re-explain changed callables, which keeps repeat runs fast and cheap.

07 · EXPLANATIONS

The Three-Lens explanation system

Each node can be read three ways. The third lens is the product's signature: it teaches the vocabulary instead of hiding it.

7.1 The three lenses

LENS	AUDIENCE AND RULES
1 · Technical	For engineers. Precise terminology, inputs and outputs, side effects, error behavior, complexity where relevant, and the role the function plays in its call path.
2 · Everyday	For anyone. No jargon. Must include at least one concrete example ("if a user types…") and one analogy. Analogies come from a single *metaphor world* chosen per repository, so explanations stay consistent across the whole map.
3 · Mapped	A bridge. The everyday explanation, rewritten so that every plain phrase is paired 1:1 with the exact technical term, symbol, or line it stands for. Tapping either side highlights its twin.

7.2 Worked example

Function: `auth.verify_token(token: str) -> User` in a FastAPI app. Metaphor world for this repository: *a hotel*.

LENS 1 · TECHNICAL

Decodes a JWT with HS256 using the `JWT_SECRET` environment variable, validates `exp` and `iss` claims, then loads the user by the `sub` claim through `UserRepo.get()`. Raises `HTTPException(401)` on any decode or lookup failure. Called by the `get_current_user` dependency on every protected route.

LENS 2 · EVERYDAY

This is the front desk checking a room key. When someone tries to enter a members-only part of the app, they show a key card. The desk checks that the card was made by this hotel, hasn't expired, and belongs to a real guest. If anything is off, the guest is turned away. For example, a card issued yesterday that was only good for one hour gets rejected.

LENS 3 · MAPPED

the front desk	↔ `verify_token()`
a key card	↔ `token: str (a JWT)`
made by this hotel	↔ `signature check, HS256 + JWT_SECRET`
hasn't expired	↔ `exp claim validation`
belongs to a real guest	↔ `UserRepo.get(sub)`
turned away	↔ `raise HTTPException(401)`
members-only part of the app	↔ `routes using get_current_user`

7.3 Keeping the mapping honest

A plain-English layer is only valuable if it's anchored to reality. Lantern enforces that mechanically:

- **Structured output.** Models return a JSON object, not free text (schema below). Each Lens 3 pair carries an `anchor`: a symbol ID from the graph, a source line range, or a vocabulary term from a curated glossary.
- **Anchor validation.** Every anchor is checked against the graph and the source. Pairs with anchors that don't exist are rejected, and the explanation is regenerated or flagged. A model cannot invent a function that isn't there.
- **Grounding context.** The prompt includes the callable's source, its callers and callees from the graph, any security findings on it, and the repository's metadata. Models explain what the graph shows, not what they guess.
- **Glossary build-up.** Technical terms that recur across a repository (JWT, middleware, mutex) gain a glossary entry in all three lenses, so a learner can browse vocabulary on its own.

```
// Explanation record (abridged)
{
  "node_id": "py:auth.verify_token",
  "code_hash": "b3f1…9a",  "model": "provider/model-name",  "prompt_version": "lens-v3",
  "lens1": { "summary": "…", "inputs": [ … ], "side_effects": [ … ], "errors": [ … ] },
  "lens2": { "text": "…", "example": "…", "analogy": "…", "metaphor_world": "hotel" },
  "lens3": [ { "plain": "the front desk", "technical": "verify_token()",
               "anchor": { "type": "symbol", "id": "py:auth.verify_token" } }, … ],
  "validated": true
}
```

7.4 Generation strategy

- **Bottom-up, then top-down.** Leaf functions are explained first, so callers can reference what their callees do. Module and entry-point summaries are generated last from their members' explanations.
- **Lazy by default.** Entry points and their first three levels of calls are explained eagerly. Everything else is explained when first opened, which keeps the first run's cost low. "Explain everything" is one tap, with a cost estimate first.

- **Path narratives.** In Flow view, the user can ask for a step-by-step story of one entry point's path ("what happens when someone logs in"), in any lens.

08 · MODELS

Model choice: the multi-LLM layer

Users choose the model for each job, see what it will cost before it runs, and can always tell which model wrote what.

8.1 Provider support

- **Bring your own key (BYOK)** for major providers: Anthropic, OpenAI, Google, Mistral, and aggregators such as OpenRouter that expose many models behind one key.
- **Local and self-hosted models** through any OpenAI-compatible endpoint, which covers Ollama, llama.cpp's server, vLLM, and LM Studio.
- **Adapter design.** A thin internal interface with three adapters (OpenAI-compatible, Anthropic, Google) covers nearly every provider. The cloud backend MAY use a gateway library such as LiteLLM for routing, spend tracking, and fallbacks; the agent ships its own small adapters so it stays a single dependency-free binary.
- **Capability checks.** Before a model is selectable for a task, Lantern runs a short probe for structured JSON output and context length. Models that fail the Lens 3 schema reliably are marked "not recommended for Mapped lens."

8.2 Per-task routing

TASK	WHAT IT NEEDS	TYPICAL CHOICE
Lens 1–3 explanations	High volume, structured output, good writing	A fast, inexpensive model
Path narratives and module summaries	Long context, synthesis	A mid-tier model
Security triage and fix drafting	Careful reasoning, low error tolerance	The strongest model available
Systems-design advisor	Broad knowledge, judgment	The strongest model available

Users can accept a recommended routing preset (Economy, Balanced, Best) or set each task by hand.

8.3 Cost, caching, and transparency

- **Cost preview.** Before any model run, Lantern estimates tokens from the graph's size and shows a price range per provider. Users can set a hard spend cap per run and per month.
- **Cache key.** `(code_hash, context_hash, model, prompt_version, lens)`. An unchanged function is never paid for twice. Changing models doesn't discard the cache; both versions are kept.
- **Attribution.** Every explanation, triage note, and fix shows the model that produced it and the date. Reports list all models used.

- **Compare mode.** Any node can be explained by two models side by side. This doubles as a quality check and helps users pick a default.

8.4 Key custody

MODE	WHERE KEYS LIVE
Local agent	The OS keychain (macOS Keychain, Windows Credential Manager, Secret Service on Linux). Never transmitted to Lantern.
Cloud	Envelope-encrypted: a per-user data key encrypts the API key; a managed KMS key encrypts the data key. Decrypted only inside the worker for the job's duration. Never logged, never returned to any client after entry; the UI shows only the last four characters.

09 · SECURITY

Security audit

Scanners find; the call graph prioritizes; models explain and draft fixes; a rescan verifies. Each step is labeled so an engineer can see where every claim came from.

9.1 Scanner stack

CONCERN	PYTHON	TYPESCRIPT / JAVASCRIPT	RUST	C++
Code patterns and taint	Opengrep or Semgrep CE engine; Bandit	Opengrep or Semgrep CE engine; ESLint security plugins	Opengrep or Semgrep CE engine; Clippy security lints	Opengrep or Semgrep CE engine; cppcheck; clang-tidy (cert-*, bugprone-*)
Vulnerable dependencies	pip-audit / OSV-Scanner	OSV-Scanner on `package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`	cargo-audit (RustSec) / OSV-Scanner	OSV-Scanner on Conan lockfiles; inventory of vendored libraries
Unsafe or risky code	Use of `eval`, `pickle`, `subprocess` with shell	`eval`, `innerHTML`, `dangerouslySetInnerHTML`, tokens in `localStorage`	cargo-geiger (count of `unsafe`)	Raw buffers, unchecked casts, banned functions list
Secrets	gitleaks across the working tree, including frontend source and `.env` files, and, in local mode, optionally across git history			
Browser exposure	Server-only secrets referenced through prefixes that bundlers ship to every visitor (`VITE_`, `NEXT_PUBLIC_`, `REACT_APP_`, `EXPO_PUBLIC_`). Ranked critical, because anyone can read them from the browser.			
Supply chain	Lockfile presence, pinned versions, typosquat heuristics, license inventory			

LICENSING: REQUIRES LEGAL REVIEW BEFORE LAUNCH

In December 2024 Semgrep moved its maintained rules to the Semgrep Rules License v1.0, which limits them to internal, non-competing, non-SaaS use. The engine itself remains LGPL 2.1. Lantern's cloud mode is a SaaS, so it MUST NOT bundle Semgrep-maintained rules there. The plan: run the Opengrep fork (or the LGPL engine) with Lantern's own rules plus permissively licensed community rules. In local mode, the agent MAY run a user's own Semgrep installation and rules, since that is the user's internal use; counsel should confirm this reading.

9.2 The finding lifecycle

1. **Collect and normalize.** All scanner output is converted to SARIF, deduplicated, and attached to graph nodes by file and line.

2. **Reachability.** Lantern checks whether the affected function is reachable from any entry point, and whether untrusted input (a route parameter, file upload, CLI argument) can flow there along graph edges. Reachable findings rank up; unreachable ones rank down but are never hidden. This is the feature engineers will value most: it separates real risk from noise.
3. **Triage.** The chosen model explains the finding in all three lenses and gives a likelihood that it's a false positive, with reasoning. Models MAY flag a likely false positive; they MUST NOT dismiss one. Only the user can mark a finding as accepted risk, with a written reason that appears in the report.
4. **Fix.** The model drafts a minimal patch as a unified diff, with a Lens 2 explanation of what changed and why.
5. **Verify.** After the patch is applied, the same scanner re-runs on the affected files. A finding is only marked "fixed" when the scanner agrees.

9.3 Severity model

RANK	RULE
Critical	High scanner severity, reachable from an entry point, and touched by untrusted input. Also any live secret.
High	High severity and reachable, or a known-exploited dependency vulnerability.
Medium	Medium severity and reachable, or high severity but unreachable.
Low	Everything else, including hardening suggestions.

9.4 Security features to add

Beyond finding bugs, Lantern recommends protections the app is missing, based on what the graph shows it does. Each recommendation names the evidence that triggered it.

IF THE GRAPH SHOWS…	LANTERN SUGGESTS
Public HTTP routes	Authentication on non-public routes, rate limiting, request size limits, security headers, CORS configured narrowly
Route input reaching a database call	Parameterized queries or an ORM, input validation schemas (Pydantic, serde with validation)
Passwords handled	A slow password hash (Argon2id or bcrypt), never a plain or fast hash
File uploads	Type and size checks, storage outside the web root, randomized file names
Secrets read from code or config files	Environment variables or a secrets manager, plus a committed `.env.example`

IF THE GRAPH SHOWS…	LANTERN SUGGESTS
Frontend calls reaching a route with no login check on its path	An auth dependency or middleware on the route, or an explicit "public" marker so the choice is deliberate
Server secrets referenced from frontend code	Move the call behind a backend route that holds the key; rotate the exposed key
A hosted database client (Supabase, Firebase) used from the browser	Row-level security or security rules turned on for every table; service-role keys kept server side only
Cross-origin requests from the frontend	CORS limited to the app's own origins, never a wildcard with credentials
Outbound calls built from user input	An allow-list to prevent server-side request forgery
C++ or `unsafe` Rust handling external input	Bounds-checked containers, sanitizer builds (ASan, UBSan) in CI, fuzzing harness for parsers

10 · READINESS

Deployment readiness and systems design

A score that can be gamed is worthless. So most checks are deterministic, every point has a visible reason, and the advisor produces real files rather than advice alone.

10.1 The Deployment Readiness Score

Scored 0–100 across ten weighted categories. Weights below are a starting proposal to validate with reviewing engineers in Phase 1.

CATEGORY	WEIGHT	EXAMPLE CHECKS
Security findings	20	No open Critical or High findings; accepted risks documented
Secrets and config	12	No secrets in code; config from environment; `.env.example` present
Input and error handling	10	Validation at boundaries; no bare `except:`; no `unwrap()` on external input; errors don't leak stack traces to users
Dependencies	10	Lockfile committed; no known-vulnerable versions; versions pinned
Tests	10	Tests exist; tests touch every entry point; tests pass (local mode, or opt-in sandbox)
Observability	8	Structured logging; a health-check endpoint for services; no `print` debugging on hot paths
Packaging	8	Reproducible build; container definition with a non-root user and a slim base image
CI/CD	8	A CI workflow that builds, tests, and scans on every push
Resilience	8	Timeouts on outbound calls; retries with backoff; graceful shutdown
Documentation	6	README with run instructions; SECURITY.md; license

10.2 Systems-design advisor

The advisor classifies the project's shape from the graph (web API, background worker, CLI tool, library, desktop app, data pipeline), then recommends a deployment pattern and explains it in all three lenses.

- **Shape-specific advice.** A web API gets twelve-factor configuration, a container, a managed database, a reverse proxy, and horizontal scaling notes. Long-running request handlers get a recommendation to move work onto a queue. A CLI gets release packaging and cross-platform builds.

- **Hotspot detection.** The graph reveals fan-in hotspots (one function everything depends on), synchronous network calls inside loops, and database calls inside request loops (the N+1 pattern). Each is flagged with the path that shows it.
- **Generated starter files.** Dockerfile, `.dockerignore`, a GitHub Actions workflow that runs tests and Lantern's scanners, `.env.example`, SECURITY.md, and a health-check handler. Each is delivered as a diff the user reviews; nothing is committed silently.
- **Architecture diagram.** A one-page diagram of the recommended deployment, which can be included in the Evidence Report.

11 · EVIDENCE

The Evidence Report

The deliverable that changes the conversation with a skeptical engineer: short, verifiable, and honest about its limits.

11.1 Contents

1. **Header.** Repository, commit hash, branch, analysis date, mode (cloud or local), and privacy level.
2. **Scorecard.** Readiness score with category breakdown; finding counts by severity, before and after fixes.
3. **Architecture.** Entry points, the top-level Atlas, and the recommended deployment diagram.
4. **Security.** Open findings, fixed findings with verification status, and accepted risks with the builder's written reasons.
5. **Understanding.** The builder can attach path narratives they've reviewed, demonstrating they know how key flows work.
6. **Provenance.** Every tool and version used, every model used, and the rule sets applied.
7. **Limits.** A required, plain-language section stating what the report does not prove, such as runtime behavior not covered by tests, business-logic flaws scanners can't see, and lower accuracy for dynamic code or C++ analyzed without a compile database.

11.2 Verifiability

- Each report gets a public, unguessable link and a PDF export. The builder can revoke the link at any time.
- The report includes a signed digest of its results and the commit hash. A reviewer can re-run the same analysis on that commit with the open-source agent and compare.
- A README badge can show the latest score, linked to the full report.

> **WHY OPEN-SOURCE THE AGENT**
> An engineer who doubts the report can read the code that produced it and run it themselves. That is the strongest possible answer to "this is just AI output." The commercial value sits in the hosted experience, the explanations, the PWA, and team features, not in the scanner orchestration.

12 · CLIENT

The PWA client

One installable web app for phone, tablet, and desktop. It is the control panel and the viewer, never the analysis engine.

12.1 Screens

SCREEN	PURPOSE
Home	Repositories, latest score per repo, runs in progress, connected agents and their online status
Atlas · Network	Whole-repo map with module clustering, search, and confidence-styled edges
Atlas · Flow	One entry point's call path as a top-down flowchart; tap a node to open its detail sheet
Node detail	Lens toggle (1 · 2 · 3), source panel, callers and callees, findings on this node, model attribution
Findings	Ranked list with filters; finding detail with lenses, reachability path, diff, and verify status
Readiness	Score breakdown, checklist, advisor output, generated files
Reports	Create, preview, publish, revoke
Settings	Models and keys, routing presets, spend caps, privacy level, connected repos and devices

12.2 Rendering graphs on every screen size

- **Network view** uses WebGL rendering (Sigma.js on a Graphology model) so tens of thousands of nodes stay smooth, with level-of-detail labels.
- **Flow view** uses a layered layout engine (ELK via elkjs) to draw a clean top-down flowchart of one path.
- **On phones**, Flow view defaults to a vertical "path list": each step is a card, indented by call depth, with a swipe to expand callees. The full graph remains available but is never the default on a small screen.
- **Accessibility.** Every graph has an equivalent list representation for screen readers and keyboard navigation. Lens colors are never the only signal.

12.3 PWA capabilities

CAPABILITY	IMPLEMENTATION
Install	Web app manifest with icons and shortcuts; install prompt after the first completed analysis
Offline	Service worker (Workbox) caches the app shell. Graphs, explanations, and reports for recently viewed repos are stored in IndexedDB, so past analyses can be explored on a plane.

CAPABILITY	IMPLEMENTATION
Notifications	Web Push when a run finishes or a new Critical finding appears. On iPhone and iPad, web push requires the PWA to be added to the home screen; onboarding explains this.
Remote control	From the phone, trigger "analyze now" on a paired agent at home, then watch progress stream in over a WebSocket
Local-only use	In Offline privacy level, the agent serves the PWA from localhost, so no data touches Lantern's servers at all

13 · STACK

Technology stack, data model, and API

Choices favor a single portable engine, boring infrastructure, and components the team can replace later.

13.1 Proposed stack

LAYER	CHOICE	RATIONALE
Engine and agent	Rust; tree-sitter grammars compiled in; language servers and scanners run as managed subprocesses	One static binary per platform; fast; memory-safe parsing of untrusted input
Graph format	Versioned schema, serialized as compressed JSON for transport and stored in Postgres with JSONB plus an edges table	Simple to start; graph database only if query needs demand it
Backend API	Python with FastAPI, or TypeScript with Fastify; REST for CRUD, WebSocket for progress	Either is fine; pick the language the founding team ships fastest in
Jobs	Postgres-backed queue initially; move to a dedicated queue when volume requires	Fewer moving parts early
Sandbox	Firecracker microVMs or gVisor on a container platform	Strong isolation for untrusted repos
Storage	Postgres (results, metadata), object storage (reports, PDFs), managed KMS (keys)	Standard and well understood
PWA	SvelteKit or React with Vite; vite-plugin-pwa (Workbox); Sigma.js, Graphology, elkjs	Mature PWA tooling; WebGL graph rendering
Auth	GitHub OAuth plus email magic links; passkeys as an option	Matches the audience; no passwords to leak

13.2 Core entities

ENTITY	KEY FIELDS
User	id, auth identities, default routing preset, spend caps
Repo	id, owner, source (github / local), display name, privacy level, alias map location
Device	id, user, platform, agent version, last seen, token hash, revoked
Run	id, repo, commit, mode, status, stage timings, tool versions, models used, cost

ENTITY	KEY FIELDS
Graph	run, schema version, nodes, edges with confidence, entry points
Finding	id, run, node, tool, rule, SARIF blob, severity, reachability path, status, accepted-risk reason
Explanation	node fingerprint, model, prompt version, lenses, validation result
Report	id, run, public slug, signed digest, revoked

13.3 API sketch

```
POST  /v1/repos                      connect a GitHub repo or register a local one
POST  /v1/repos/{id}/runs            start a run (cloud) or ask a paired agent to run
GET   /v1/runs/{id}                  status, stages, cost so far
WS    /v1/runs/{id}/stream           live progress and streamed explanations
GET   /v1/runs/{id}/graph?entry=…    graph, optionally scoped to one entry point
GET   /v1/nodes/{fp}/explanations    all lenses, all models
POST  /v1/nodes/{fp}/explain         generate on demand with a chosen model
GET   /v1/runs/{id}/findings         ranked findings
POST  /v1/findings/{id}/fix          draft a patch; open a PR in cloud mode
POST  /v1/runs/{id}/reports          publish an Evidence Report
POST  /v1/devices/pair               exchange a one-time code for a device token
POST  /v1/agent/results              agent uploads results (signed, chunked)
```

14 · TRUST

Trust and security of Lantern itself

A product that judges other people's security will be judged on its own. This section is a launch gate, not a wish list.

14.1 Threat model highlights

THREAT	MITIGATION
Malicious repository escapes the sandbox	MicroVM isolation; no build execution by default; no credentials inside the VM beyond a short-lived model key; restricted egress
Prompt injection in source code (comments telling the model to report "no issues")	Findings come from scanners, not models, so injection cannot hide a finding. Model output is schema-validated. Source is passed as clearly delimited data. Triage cannot close findings.
Stolen API keys	Envelope encryption; keys decrypted only inside job scope; never logged; local mode keeps keys in the OS keychain
Compromised agent update	Signed releases; reproducible builds; checksums published; update channel pinned to signing key
Stolen device token	Device-scoped tokens, revocable from the PWA; agent requests are signed and rate-limited
Leaked source via reports	Privacy levels; reports default to Structure only; explicit confirmation before publishing snippets
GitHub App over-permission	Read-only contents by default; PR permission requested only on first use; per-repo selection

14.2 Launch gates

- Lantern's own repositories pass Lantern's audit with no open Critical or High findings, and the report is published.
- An independent penetration test of the sandbox and API, with a public summary.
- Published data-handling policy in plain language, written in all three lenses.
- A vulnerability disclosure policy and security contact.

15 · QUALITY

Non-functional requirements

Targets to validate with benchmark repositories in Phase 0. They are design goals, not measured results.

REQUIREMENT	TARGET
Structural analysis, 50k-line repo (Python and TS/JS, any mix)	Under 3 minutes in cloud mode, first run
Bridge accuracy	At least 95% of resolved and inferred frontend-to-backend links correct on a labeled set of full-stack repos
Incremental re-run after a small change	Under 20 seconds for stages 1–5
First explanation visible	Under 30 seconds after stage 5 completes
Atlas interaction	60 fps pan and zoom up to 20k nodes on a recent laptop; Flow view usable on a mid-range phone
Edge precision (resolved edges)	At least 95% correct on a labeled benchmark per language
Lens 3 anchor validity	100% of published pairs resolve to real symbols, lines, or glossary terms (enforced, not sampled)
Availability	99.5% for API and PWA; local mode keeps working when the cloud is down
Agent footprint	Binary under 60 MB, excluding language servers and scanners
Accessibility	WCAG 2.2 AA for all PWA screens

16 · ROADMAP

Roadmap

Each phase ends with something real users can try. Durations are estimates for a small team and will be revised after Phase 0.

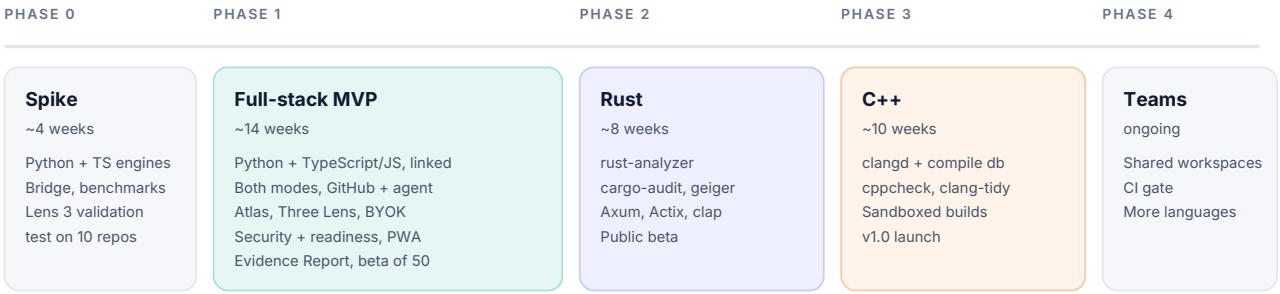


Figure 3. Phased delivery. Phase 1 is deliberately the largest: it builds every layer end to end for a full-stack app (a Python or TypeScript backend with a TypeScript frontend), so later languages slot into a finished product.

16.1 Phase 0 exit criteria and status

- Engine produces a graph for 10 public Python repositories of varying size, with edge precision measured against hand-labeled samples.
- The TypeScript/JavaScript engine and the bridge produce a linked graph for at least three full-stack repositories (for example FastAPI with React, Next.js with its own API routes, Express with a Vite frontend), with bridge links checked by hand.
- Lens 3 generation passes anchor validation on at least 90% of nodes on first attempt with two different model providers.
- Five AI-assisted builders and five working engineers react to sample Three-Lens explanations and a mock Evidence Report. Their feedback sets the readiness weights and report layout.

Measured so far (October 2026)

CHECK	RESULT
Python edge precision (resolved edges, lower bound)	95.5% to 96.7% on Click, Flask, and Rich, measured against each project's own test suite running under the tracer. Meets the 95% target.
Python speed	About 1 second for 40,000 lines, against a 3-minute target
Full stack: FastAPI template	2,900 lines of Python and 10,000 of TypeScript analyzed together in 0.3 seconds. All 23 frontend API calls linked to the correct backend route, checked by hand.

CHECK	RESULT
Still open	Lens 3 first-try pass rate with real models; a JavaScript runtime tracer so TypeScript precision is measured the way Python's is; more full-stack benchmark repositories

16.2 Business model sketch

TIER	OFFER
Free	Local agent, unlimited local analysis, BYOK or local models, one public Evidence Report
Pro	Cloud mode, private reports, unlimited reports, PR fixes, priority runs
Team	Shared workspaces, CI readiness gate, onboarding maps, admin controls

Users always pay model providers directly through their own keys in v1, so Lantern carries no model-cost risk. Hosted model credits are a later option.

17 · RISKS AND METRICS

Risks, open questions, and success metrics

The biggest risks are trust risks. Most of the design above exists to reduce them.

17.1 Risks

RISK	IMPACT	MITIGATION
Graphs are wrong for dynamic Python, hook-heavy React, or macro-heavy C++	High	Confidence levels on every edge; runtime tracing; compile databases; honest Limits section
The bridge links a frontend call to the wrong route	Medium	Method and path must both match; prefix matches are shown as likely, ties as possible; hand-labeled full-stack benchmark; runtime tracing confirms links when available
Engineers dismiss the report as AI output	High	Scanner-sourced findings, provenance, open-source agent, reproducible results
Analogies mislead learners	Medium	Lens 3 anchors; consistent metaphor worlds; "this analogy breaks down when..." notes; user feedback per explanation
Model costs surprise users	Medium	Cost preview, spend caps, lazy explanation, aggressive caching
Scanner rule licensing	Medium	Opengrep plus own and permissive rules in cloud; legal review before launch
Sandbox escape from a hostile repo	High	MicroVMs, no build execution by default, pen test as a launch gate
A large incumbent ships something similar	Medium	Focus on the underserved audience; the Mapped lens and Evidence Report as differentiators; community around the open agent

17.2 Open questions

1. Final product name, plus trademark and domain availability.
2. Backend language: Python or TypeScript. Decide by team skills in Phase 0.
3. Should metaphor worlds be chosen by the user, by the model per repository, or from a curated set?
4. Open-source license for the agent: permissive (Apache 2.0) or copyleft (AGPL)?
5. Should the Evidence Report support a reviewer sign-off, letting an engineer attest they reviewed it?
6. Which second code host to add after GitHub, if any, for cloud mode?
7. Which hosted backends to model first as database edges with access-rule checks: Supabase, Firebase, or both?

8. Should Lantern read a backend's OpenAPI document, when one exists, to turn prefix matches into exact ones?

17.3 Success metrics

METRIC	WHY IT MATTERS
Median readiness score gain between a repo's first and fifth run	Shows the product actually improves code, not just describes it
Critical and High findings fixed and verified per active repo	The core safety outcome
Share of users who toggle into Lens 1 after starting in Lens 2	Shows the Mapped lens is working as a learning ramp
Evidence Reports opened by someone other than the author	Measures the product's reach into the engineer audience
Explanation thumbs-down rate, per model and lens	Quality signal for routing defaults and prompt changes
Split of runs between cloud and local mode	Validates the two-mode investment and guides roadmap

> **RECOMMENDED NEXT STEP**
> Finish Phase 0. The Python engine, the TypeScript/JavaScript engine, and the bridge are built and benchmarked. What remains: run the Lens 3 validator with real models from two providers, add full-stack benchmark repositories, and put sample reports in front of five builders and five engineers. That answers the two questions that decide everything else: can the graph be trusted across the whole stack, and does the Mapped lens actually teach.